Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 9

Disciplina: Tehnici de programare

Tema: ” Colecții: liste, tupluri, mulțimi, dicționare”

A efectuat : st. gr.SI – 261 Talchiv Corina

A verificat : asistent universitar Dumitrașcu Marius

Chișinău – 2026

SARCINA LUCRĂRII

Se construiește o unealtă de enumerare spre root, enum.py, care citește ce a adunat un atacator pe o mașină Linux compromisă (fișiere SUID, reguli sudo, sarcini cron), așază constatările în colecțiile Python potrivite și produce un raport JSON (**Lucrarea A**). Apoi se compară sistemul cu o imagine curată și se scot, cu operații pe mulțimi și cu comprehensions, exact fișierele SUID noi și periculoase (**Lucrarea B**)

Fișier	Conținut
arsenal/privesc/enum.py	Citește capturile. Grupează pe categorii,scoate rezumat și JSON
enum.json	Raportul generat:dicțtionar de liste de dicționare
arsenal/privesc/diferenta.py	Diferența de mulțimi față de sistemul curat
arsenal/privesc/test_diferenta.py	Un test care trece cu pytest

Scopul și obiectivele lucrării

Scopul: însușirea practică a celor patru colecții fundamentale din Python și a alegerii corecte a colecției pentru fiecare tip de date, pe un scenariu real de securitate informațională (escaladarea privilegiilor pe Linux).

Obiective:

1. să citesc și să curăț date dintr-un fișier text într-o **listă**;
2. să elimin dublurile și să obțin valori distincte cu o **mulțime** (set comprehension);
3. să modelez o constatare ca **dicționar** și un set de constatări ca **listă de dicționare**;
4. să despachetez o linie într-un **tuplu** (user, drept = ...);
5. să grupez întregul raport într-un **dicționar** cu o cheie pe categorie și să-l export în **JSON**;
6. să aplic **operatori pe mulțimi** (reuniune |, diferență -) pentru a compara două stări ale sistemului;
7. să folosesc **list / dict / set comprehensions** pentru filtrare și transformare;
8. să scriu un **test automat** cu pytest care prinde regresiile;
9. să folosesc argparse, .get(), os.path.join și să predau lucrarea prin **git**.

Implementarea

Importuri și constanta GTFO (mulțime)

```
import argparse
import json
import os

GTFO = {"find", "vim.basic", "vim", "nmap", "cp", "bash",
        "less", "awk", "python3", "perl"}
```

GTFO este o **mulțime** (set) de binare cunoscute ca „scăpări” (stil GTFOBins), adică programe care, având bit SUID sau drepturi sudo, pot da shell de root. S-a ales mulțimea pentru că ordinea nu contează, elementele sunt unice, iar testul de apartenență x in GTFO este foarte rapid, în medie O(1), spre deosebire de o listă, unde e O(n).

citeste_linii(cale) (listă)

```
citeste_linii(cale) (listă)
def citeste_linii(cale):
    with open(cale, encoding="utf-8") as f:
        return [linia.strip() for linia in f
                if linia.strip() and not linia.strip().startswith("#")]
```

Funcția comună de citire întoarce o **listă** de linii curate: fără spații la capete (strip), fără linii goale și fără comentarii (#). Lista este potrivită aici pentru că păstrează **ordinea** și **dublurile** din fișier (de aceea suid.txt dă 14 linii, deși doar 13 sunt distincte). Instrucțiunea with închide fișierul automat, iar encoding="utf-8" evită erorile de codare.

nume(cale) (șir de caractere)

```
def nume(cale):
    return cale.rsplit("/", 1)[-1]
```

rsplit("/", 1) împarte calea o singură dată, de la dreapta, iar [-1] ia ultimul element. Exemplu: "/usr/bin/find" → "find". Numele programului (nu calea) decide dacă fișierul este periculos.

suid_constatari(cale) (mulțime → listă sortată → listă de dicționare)

```
def suid_constatari(cale):
    cai_unice = sorted(list(set(citeste_linii(cale))))
    return [
        {
            "cale": c,
            "nume": nume(c),
            "risc": "mare" if nume(c) in GTFO else "normal"
        }
        for c in cai_unice ]
```

Funcția îmbină trei colecții: set(...) elimină dublurile, sorted(...) aduce rezultatul într-o ordine stabilă (o mulțime nu are ordine garantată), iar un **list comprehension** construiește câte un **dicționar** {"cale", "nume", "risc"} pentru fiecare cale unică. Expresia condițională "mare" if ... else "normal" stabilește riscul.

sudo_constatari(cale) (tuplu și listă de dicționare)

```
def sudo_constatari(cale):
    out = []
    for ln in citeste_linii(cale):
        user, drept = ln.split(" : ", 1)      # despachetare în tuplu
        out.append({
            "user": user.strip(),
            "drept": drept.strip(),
            "risc": "mare" if "NOPASSWD" in drept else "normal"
        })
    return out
```

Linia ana : (ALL) NOPASSWD: /usr/bin/vim.basic este împărțită în două cu split(" : ", 1). Rezultatul este o pereche, iar atribuirea user, drept = ... este **despachetare de tuplu** (tuple unpacking). Parametrul 1 limitează împărțirea la prima apariție a separatorului, deoarece în partea de drept pot apărea și alte caractere similare. Regula este marcată cu risc „mare” dacă conține NOPASSWD, adică poate fi rulată ca root fără parolă.

cron_constatari(cale) (listă de dicționare)

```
def cron_constatari(cale):
    out = []
    for ln in citeste_linii(cale):
        campuri = ln.split()
        if len(campuri) >= 7:
            user = campuri[5]
            comanda = " ".join(campuri[6:])
            out.append({"user": user, "comanda": comanda})
    return out
```

O linie /etc/crontab are 5 câmpuri de program (minut, oră, zi, lună, zi a săptămânii), apoi utilizatorul (indexul 5) și comanda (de la indexul 6, reunită cu " ".join(...) fiindcă poate conține spații). Condiția len(campuri) >= 7 este o **protecție** adăugată față de schelet: o linie scurtă sau malformată este ignorată, în loc să provoace IndexError.

enumereaza(d) (dicționar de liste)

```
def enumereaza(d):
    return {
        "suid": suid_constatari(os.path.join(d, "suid.txt")),
        "sudo": sudo_constatari(os.path.join(d, "sudo.txt")),
        "cron": cron_constatari(os.path.join(d, "crontab.txt")),
    }
```

Raportul final este un **dicționar** cu o cheie pe categorie, iar valoarea fiecărei chei este o listă de dicționare. os.path.join construiește căile independent de sistemul de operare. Datele nu sunt ținute în variabile separate, ci într-o singură structură, ușor de parcurs și de serializat în JSON.

main() (argparse, .items(), .get(), reuniune de mulțimi)

```
def main():
    p = argparse.ArgumentParser()
    p.add_argument("capturi")
    p.add_argument("--json", action="store_true")
    a = p.parse_args()

    raport = enumereaza(a.capturi)
    if a.json:
        print(json.dumps(raport, indent=2, ensure_ascii=False))
        return

    for categorie, lista in raport.items():
        mari = [c for c in lista if c.get("risc") == "mare"]
        print(f"{categorie}: {len(lista)} gasite, {len(mari)} cu risc mare")

    useri_sudo = {item["user"] for item in raport["sudo"]}
    useri_cron = {item["user"] for item in raport["cron"]}
    utilizatori_implicati = useri_sudo | useri_cron
    print(f"utilizatori implicati: {', '.join(sorted(utilizatori_implicati))}")

    with open("enum.json", "w", encoding="utf-8") as f:
        json.dump(raport, f, indent=2, ensure_ascii=False)
    print("Raport salvat in enum.json")
```

Lucrarea B: diferenta.py

```
import os
from arsenal.privesc.enum import citeste_linii, nume, GTFO

DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "start")

def cai(fisier):
    return set(citeste_linii(fisier))

baza = cai(os.path.join(DIR, "baza_suid.txt"))
acum = cai(os.path.join(DIR, "suid.txt"))
print("baza:", len(baza), " acum:", len(acum))

noi = acum - baza
print(len(noi), "SUID noi fata de sistemul curat")
for c in sorted(noi):
    print(" ", c)

risc = {nume(c): ("mare" if nume(c) in GTFO else "normal") for c in noi}
periculoase = [c for c in sorted(noi) if nume(c) in GTFO]
print("periculoase NOI:", periculoase)
```

Testul test_diferenta.py

```
def test_doar_ce_e_nou():
    acum = {"a", "b", "c"}
```

```
baza = {"a", "b"}
assert acum - baza == {"c"}
```

CONCLUZII

În această lucrare am construit o unealtă de enumerare pentru escaladarea privilegiilor pe Linux și am folosit toate cele patru colecții fundamentale din Python, fiecare acolo unde se potrivește natural:

- **lista** a păstrat liniile fișierelor în ordine, inclusiv dublurile, și a ținut seturile de constatări;
- **tuplul** (prin despachetare) a descompus eficient liniile din sudo.txt;
- **mulțimea** a eliminat dublurile (14 linii → 13 nume distincte), a testat rapid apartenența la GTFO și a permis comparații prin operatori (-, |);
- **dicționarul** a modelat fiecare constatare și întregul raport, care a devenit direct un JSON valid.

Am văzut că alegerea corectă a structurii de date face codul mai scurt și mai clar: diferența dintre sistemul curat și cel compromis se reduce la un singur operator (acum - baza), iar filtrarea periculoaselor la un comprehension cu if. Rezultatul practic: pe sistemul analizat au apărut **4 fișiere SUID noi** (cp, find, nmap, vim.basic), toate cunoscute ca scăpări către root, la care se adaugă două reguli sudo NOPASSWD (vim.basic pentru ana, find pentru www-data) care permit același lucru. Împreună, aceste constatări indică mai multe căi directe spre root.

Pe lângă partea de colecții, am exersat argparse, .get() pentru chei care pot lipsi, scrierea și citirea JSON, testarea automată cu pytest și lucrul cu pachete și importuri. Problemele întâlnite (căi relative, suprapunerea numelui enum cu modulul standard) m-au învățat că rularea corectă a codului (python -m din rădăcină) și căile calculate față de fișier sunt la fel de importante ca logica programului.